

Universidad Nacional del Nordeste

Facultad de Ciencias Exactas y Naturales y Agrimensura

Licenciatura en Sistemas de Información

Métodos Computacionales

Grupo N°9

Integrantes	LU
Mazzanti, Marcos Santino	60140
Méndez, Nazareno Joaquin	60107

Métodos de Interpolación Lineal y Newton Raphson

Año 2026 – Lab. Jueves(15hs)

Índice de contenidos

Introducción....4

Desarrollo....5

Métodos asignados. Análisis y comparación....5

Código Fuente....7

Resultado de la ejecución....19

Gráficos....27

Conclusión....29

Bibliografía....30

Índice de figuras

Figura 1: Método de tanteos Ejercicio 1.....20

Figura 2: Resultado Interpolación Lineal.....21

Figura 3: Resultado Newton Raphson.....21

Figura 4: Resultados finales y comparación ejercicio 1.....22

Figura 5: Método de tanteos Ejercicio 2.....23

Figura 6: Interpolación Lineal - Intervalo [250, 260].....24

Figura 7: Interpolación Lineal - Intervalo [20, 30].....24

Figura 8: Newton Raphson - Intervalo [250, 260].....25

Figura 9: Newton Raphson - Intervalo [20, 30].....25

Figura 10: Ejercicio 2 - Resultados finales y comparación intervalo [250, 260].....26

Figura 11: Ejercicio 2 - Resultados finales y comparación intervalo [20, 30].....26

Figura 12: Gráfico - Ejercicio 1.....27

Figura 13: Grafico - Ejercicio 2.....28

Introducción

En muchos problemas matemáticos o de ingeniería, el modelo termina en una ecuación $f(x) = 0$, donde despejar x con una fórmula a mano suele ser complicado o hasta imposible, sobre todo si se incluyen términos exponenciales, logarítmicos o trigonométricos. Ante esto se recurre a métodos computacionales o numéricos, que aproximan la raíz repitiendo un cálculo hasta obtener un resultado satisfactorio o que cumpla cierto criterio.

Durante este informe se abordan dos métodos, Interpolación Lineal y Newton-Raphson. Si bien se aplican de forma diferente, comparten pasos previos propios de este tipo de métodos: primero se busca un intervalo amplio en el cual se espera que estén las raíces; luego se lo divide en intervalos más reducidos que aíslan cada raíz; y por último se aplican algoritmos iterativos que se aproximan al valor numérico final y lo van refinando. La diferencia entre ambos está en esta última etapa.

Para aislar las raíces se usa el método de tanteo junto con el Teorema de Bolzano ($f(a)\cdot f(b) < 0$). Este establece que si una función continua cambia de signo en los extremos de un intervalo, dicho intervalo contiene como mínimo una raíz; esa raíz es única si además la función es monótona en el tramo. Como estos métodos generan sucesiones de valores que en teoría no terminan, se implementan condiciones de parada: un máximo de iteraciones (fijado en 100) o una tolerancia al error, medida como la diferencia absoluta entre dos aproximaciones sucesivas.

El objetivo es comparar ambos métodos y analizar si existen contextos en los que convenga más uno que otro. Todos los algoritmos utilizados se presentan en GNU Octave, que ofrece herramientas para el cálculo numérico.

Desarrollo

Métodos asignados. Análisis y comparación

Interpolación Lineal

Es un método cerrado (también conocido como regla falsa o falsa posición): parte de un intervalo [a, b] donde la función f es continua y cumple $f(a)\cdot f(b) < 0$, de modo que, por el Teorema de Bolzano, la raíz queda encerrada en él. En lugar de dividir el intervalo a la mitad, como hace el método del intervalo medio, reemplaza la curva por la recta secante que une los puntos (a, f(a)) y (b, f(b)), y toma como aproximación el punto donde esa recta corta al eje x. Igualando a cero la ecuación de la recta se obtiene:

$$x_{n+1} = \frac{a\ f(b) - b\ f(a)}{f(b) - f(a)}$$

Luego se evalúa $f(x_{n+1})$ y se conserva el subintervalo donde sigue habiendo cambio de signo: si $f(a)\cdot f(x_{n+1}) < 0$ la raíz está en $[a, x_{n+1}]$, y en caso contrario en $[x_{n+1}, b]$. Si $f(x_{n+1}) = 0$, se encontró la raíz exacta y el proceso termina.

Como el cambio de signo se conserva en cada paso, el método converge siempre que f sea continua en el intervalo. A cambio, su convergencia es lineal: el error se reduce aproximadamente en una proporción constante en cada iteración. Cuando la función es muy curva en el intervalo, uno de los extremos tiende a quedar fijo y el avance se vuelve lento. Su ventaja es que solo necesita evaluar f, sin derivadas.

Newton-Raphson

Es un método abierto: en lugar de un intervalo, parte de un único punto inicial x_0. En cada paso reemplaza la curva por la recta tangente en $(x_n, f(x_n))$, y la nueva aproximación es el punto donde esa tangente corta al eje x:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Para que funcione, f debe ser derivable y $f'(x_n)$ no puede anularse, porque una tangente horizontal nunca corta al eje. La elección de x_0 es clave: si la raíz está aislada en [a, b] y f′

y f″ no cambian de signo en ese tramo, la condición de Fourier indica tomar como punto inicial el extremo donde se cumple $f(x_0)\cdot f''(x_0) > 0$, con lo cual la convergencia queda asegurada. Un mal punto inicial puede hacer que el método se aleje de la raíz u oscile.

Comparación

	Interpolación Lineal	**Newton-Raphson**
Tipo de método	Cerrado	Abierto
Dato inicial	Intervalo [a, b] con cambio de signo	Un punto X_0
Evalúa en cada paso	f(x)	f(x) y f'(x)
Convergencia	Garantizada si f es continua en [a,b]	Condicionada a X_0 y a f'(Xn) ≠ 0
Velocidad	Lineal	Cuadrática
Ventaja principal	Seguridad, sin derivadas	Pocas iteraciones
Desventaja principal	Puede ser lento	Exige la derivada y un buen X_0

Ambos métodos parten de una misma premisa: antes de calcular hay que ubicar dónde empezar. Pero a la hora de calcular se diferencian bastante. Interpolación Lineal es un método cerrado, por lo que necesita dos puntos iniciales [a, b] entre los cuales haya una raíz encerrada, algo que se verifica con el cambio de signo de la función. Newton-Raphson, en cambio, necesita un solo punto inicial, pero en cada iteración debe calcular la primera derivada f'(x), lo que aumenta el costo de cada paso.

Si lo que se busca es una alta seguridad de convergencia, el método a elegir es Interpolación Lineal: como en cada iteración conserva el subintervalo donde la función cambia de signo, la raíz siempre queda encerrada. Newton-Raphson, en cambio, puede alejarse de la raíz (o incluso saltar a otra) si el punto inicial es malo, y también tiene problemas si f'(x) se acerca a 0, porque la tangente queda casi horizontal y el siguiente punto cae muy lejos.

En cuanto a la velocidad de convergencia, Newton-Raphson, cuando converge, lo hace más rápido que Interpolación Lineal (cuadrática contra lineal). Sin embargo, esto no

implica que haga menos trabajo computacional: en cada iteración, Interpolación Lineal solo necesita evaluar f en el punto nuevo, mientras que Newton-Raphson evalúa $f(x_i)$ y además $f'(x_i)$. Si la derivada es costosa de calcular, la ventaja de necesitar menos iteraciones puede reducirse o desaparecer.

En base a lo anterior, qué método usar depende del contexto del problema. Si se dispone de un intervalo que cumple la condición de cambio de signo y se prioriza la seguridad, conviene Interpolación Lineal. Si, en cambio, se tiene un buen punto inicial, la derivada es fácil de obtener y se busca llegar a la solución en pocas iteraciones, conviene Newton-Raphson.

Código Fuente

Presenta el código principal del cual se obtienen las salidas presentadas en el informe, junto con el código utilizado para graficar las funciones, realizar el método de tanteos y los métodos de Interpolación lineal y Newton-Raphson.

Ejecutar_metodos.m

```
%
================================================================================
==
% EJEMPLOS PARA COPIAR Y PEGAR:
%
% 1) Problema del tanque esferico (TP3 Ejercicio 1):
%    f   = @(h) pi * h.^2 .* (9 - h) / 3 - 30;
%    df  = @(h) 6*pi*h - pi*h.^2;
%    d2f = @(h) 6*pi - 2*pi*h;
%
% 2) Problema de produccion (TP3 Ejercicio 2):
%    f   = @(x) 8*x + 0.3*x.^2 - 0.0013*x.^3 - 372;
%    df  = @(x) 8 + 0.6*x - 0.0039*x.^2;
%    d2f = @(x) 0.6 - 0.0078*x;
%
================================================================================
==

clear; clc; close all;

% Rutas para dependencias (Interpolacion Lineal y Newton-Raphson)
carpeta = fileparts(mfilename('fullpath'));
addpath(carpeta);
addpath(fullfile(carpeta, '..'));
addpath(fullfile(carpeta, 'Interpolacion Lineal'));
addpath(fullfile(carpeta, 'Newton Raphson'));
```

```
fprintf('Comparacion de Metodos: Interpolacion Lineal y Newton-
Raphson\n\n');

%
--------------------------------------------------------------------------
--
% SELECCION O INGRESO DE LA FUNCION
%
--------------------------------------------------------------------------
--
fprintf('Seleccione el problema a analizar:\n');
fprintf('  1) Tanque esferico [TP3 Ej. 1]: f(h) = pi*h^2*(9-h)/3 - 30\n');
fprintf('  2) Produccion [TP3 Ej. 2]:      f(x) = 8*x + 0.3*x^2 - 0.0013*x^3
- 372\n');
fprintf('  3) Ingresar funcion personalizada por teclado\n\n');

opc_f = input('Opcion [1-3, por defecto 1]: ');
if isempty(opc_f)
  opc_f = 1;
endif

switch opc_f
  case 1
    f   = @(h) pi * h.^2 .* (9 - h) / 3 - 30;
    df  = @(h) 6*pi*h - pi*h.^2;
    d2f = @(h) 6*pi - 2*pi*h;
    nombre_var = 'h';
    x_min_def = -3;
    x_max_def = 10;
    dx_def = 1;
    es_tanque = true;

  case 2
    f   = @(x) 8*x + 0.3*x.^2 - 0.0013*x.^3 - 372;
    df  = @(x) 8 + 0.6*x - 0.0039*x.^2;
    d2f = @(x) 0.6 - 0.0078*x;
    nombre_var = 'x';
    x_min_def = 0;
    x_max_def = 300;
    dx_def = 25;
    es_tanque = false;

  case 3
    fprintf('\n');
    str_in = input('Ingrese f(x) (ej: 8*x + 0.3*x^2 - 372): ', 's');
    if isempty(str_in)
      f   = @(h) pi * h.^2 .* (9 - h) / 3 - 30;
      df  = @(h) 6*pi*h - pi*h.^2;
      d2f = @(h) 6*pi - 2*pi*h;
```

```
      nombre_var = 'h';
      x_min_def = -3;
      x_max_def = 10;
      dx_def = 1;
      es_tanque = true;
    else
      if ~strncmp(strtrim(str_in), '@', 1)
        str_in = regexprep(str_in, '(?<!\.)\^', '.^');
        str_in = regexprep(str_in, '(?<!\.)\*', '.*');
        str_in = regexprep(str_in, '(?<!\.)\/', './');
        str_in = ['@(x) ' str_in];
      endif
      f = str2func(str_in);

      % Derivadas numericas para Newton-Raphson si no se ingresan analiticas
      h_step = 1e-6;
      df  = @(x) (f(x + h_step) - f(x - h_step)) / (2 * h_step);
      d2f = @(x) (f(x + h_step) - 2*f(x) + f(x - h_step)) / (h_step^2);

      nombre_var = 'x';
      x_min_def = -10;
      x_max_def = 10;
      dx_def = 1;
      es_tanque = false;
    endif

  otherwise
    error('Opcion no valida.');
endswitch

%
-------------------------------------------------------------------------------
--
% PASO 1: GRAFICO DE LA FUNCION
%
-------------------------------------------------------------------------------
--
fprintf('\n-----------------------------------------------------------------\n');
fprintf('PASO 1: GRAFICO DE LA FUNCION\n');
fprintf('-----------------------------------------------------------------\n');

prompt_xmin = sprintf('Extremo inferior para el grafico [%g]: ', x_min_def);
xmin_in = input(prompt_xmin);
if isempty(xmin_in), x_min = x_min_def; else, x_min = xmin_in; endif

prompt_xmax = sprintf('Extremo superior para el grafico [%g]: ', x_max_def);
xmax_in = input(prompt_xmax);
if isempty(xmax_in), x_max = x_max_def; else, x_max = xmax_in; endif
```

```
raices_graf = graficar_raices(f, x_min, x_max);
if ~isempty(raices_graf)
  fprintf('Raices aproximadas visualizadas en el grafico: ');
  fprintf('%.4f  ', raices_graf);
  fprintf('\n');
endif

%
-------------------------------------------------------------------------
--
% PASO 2: METODO DE TANTEO
%
-------------------------------------------------------------------------
--
fprintf('\n-----------------------------------------------------------\n');
fprintf('PASO 2: METODO DE TANTEO\n');
fprintf('-----------------------------------------------------------\n');

prompt_dx = sprintf('Incremento dx para el tanteo [%g]: ', dx_def);
dx_in = input(prompt_dx);
if isempty(dx_in), dx = dx_def; else, dx = dx_in; endif

intervalos = tanteo(f, x_min, x_max, dx);

if isempty(intervalos)
  fprintf('\nNo se detectaron intervalos con cambio de signo en [%g, %g].
\n', x_min, x_max);
  fprintf('Pruebe con un rango mayor o un paso dx menor.\n');
  return;
endif

%
-------------------------------------------------------------------------
--
% PASO 3: SELECCION DE INTERVALO
%
-------------------------------------------------------------------------
--
num_int = size(intervalos, 1);
fprintf('\n-----------------------------------------------------------\n');
fprintf('PASO 3: SELECCION DE INTERVALO\n');
fprintf('-----------------------------------------------------------\n');

if es_tanque
  fprintf('Condicion fisica (Tanque R=3): 0 <= h <= 6 (2*R)\n\n');
endif

fprintf('Intervalos detectados por el tanteo:\n');
for i = 1:num_int
```

```
  fprintf('  [%d]  [%.4f, %.4f]\n', i, intervalos(i, 1), intervalos(i, 2));
endfor
fprintf('  [0]  Ingresar intervalo manual [a, b]\n\n');

prompt_sel = sprintf('Seleccione que intervalo utilizar [1-%d o 0]: ',
num_int);
opc_sel = input(prompt_sel);
if isempty(opc_sel)
  opc_sel = 1;
endif

if opc_sel == 0
  a = input('Ingrese extremo a: ');
  b = input('Ingrese extremo b: ');
elseif opc_sel >= 1 && opc_sel <= num_int
  a = intervalos(opc_sel, 1);
  b = intervalos(opc_sel, 2);
else
  error('Seleccion no valida.');
endif

fprintf('\nIntervalo seleccionado: [%.4f, %.4f]\n', a, b);

%
-------------------------------------------------------------------------------
--
% PASO 4: CONFIGURACION DE TOLERANCIA
%
-------------------------------------------------------------------------------
--
fprintf('\n------------------------------------------------------------\n');
fprintf('PASO 4: CONFIGURACION DE TOLERANCIA\n');
fprintf('------------------------------------------------------------\n');

tol_in = input('Tolerancia de error [default 1e-4]: ');
if isempty(tol_in), tol = 1e-4; else, tol = tol_in; endif

max_iter = 100;

%
-------------------------------------------------------------------------------
--
% PASO 5: EJECUCION DE LOS METODOS
%
-------------------------------------------------------------------------------
--
fprintf('\n------------------------------------------------------------\n');
fprintf('PASO 5: EJECUCION DE METODOS\n');
fprintf('------------------------------------------------------------\n');
```

```
% 1. Metodo de Interpolacion Lineal
fprintf('\n>>> METODO 1: INTERPOLACION LINEAL <<<\n');
tic;
c0_il = cputime;
[raiz_il, iter_il] = interpolacionLineal(f, a, b, tol, max_iter);
t_il = toc;
cpu_il = cputime - c0_il;

% 2. Metodo de Newton-Raphson
fprintf('\n>>> METODO 2: NEWTON-RAPHSON <<<\n');
tic;
c0_nr = cputime;
if exist('NewtonRaphson', 'file') == 2
  [raiz_nr, iter_nr] = NewtonRaphson(f, a, b, tol);
else
  [raiz_nr, iter_nr] = NewtonRaphson_extraido(f, df, d2f, a, b, tol);
endif
t_nr = toc;
cpu_nr = cputime - c0_nr;

%
-------------------------------------------------------------------------
--
% RESULTADOS FINALES Y COMPARACION
%
-------------------------------------------------------------------------
--
fprintf('\n-----------------------------------------------------------\n');
fprintf('RESULTADOS FINALES Y COMPARACION\n');
fprintf('-----------------------------------------------------------\n');
fprintf('%-24s %-14s %-8s %-14s %-14s\n', 'Metodo', 'Raiz', 'Iter', 'tic/toc
(s)', 'cputime (s)');
fprintf('%-24s %-14.8f %-8d %-14.3e %-14.3e\n', 'Interpolacion lineal',
raiz_il, iter_il, t_il, cpu_il);
fprintf('%-24s %-14.8f %-8d %-14.3e %-14.3e\n', 'Newton-Raphson', raiz_nr,
iter_nr, t_nr, cpu_nr);

fprintf('\nDetalle:\n');
fprintf('  Variable   : %s\n', nombre_var);
fprintf('  Intervalo  : [%.4f, %.4f]\n', a, b);
fprintf('  Tolerancia : %.1e\n', tol);
fprintf('  f(Raiz IL) : %.2e\n', f(raiz_il));
fprintf('  f(Raiz NR) : %.2e\n', f(raiz_nr));
fprintf('-----------------------------------------------------------\n');
```

graficar_raices.m

```
function raices = graficar_raices(f, a, b)
  % f: funcion
  % [a b]: intervalo donde se grafica
  % raices: vector con las raíces aproximadas que se encontraron

  % --- Cálculo de raíces: cambio de signo en una grilla fina ---
  x  = linspace(a, b, 10000);
  y  = f(x);
  i  = find(y(1:end-1) .* y(2:end) < 0);
  x1 = x(i);    y1 = y(i);
  x2 = x(i+1);  y2 = y(i+1);
  raices = x1 - y1 .* (x2 - x1) ./ (y2 - y1);

  % --- Gráfico ---
  figure;
  fplot(f, [a b], 'b-');
  hold on;
  plot([a b], [0 0], 'k--'); grid on;   % eje y = 0
  plot(raices, zeros(size(raices)), 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 8);

    for k = 1:numel(raices)
    text(raices(k), 0, sprintf('  x = %.2f', raices(k)), ...
         'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'left');
         grid on;
    endfor

  xlabel('x');
  ylabel('f(x)');
  title('Búsqueda gráfica de raíces');
  hold off;
end
```

tanteo.m

```
function intervalos = tanteo(f, a, b, dx)
  % Devuelve una matriz Nx2: cada fila es un [x1 x2] con cambio de signo
  intervalos = [];


  fprintf('\n>>> BUSQUEDA DE RAICES POR TANTEO <<<\n');
  fprintf('%-10s | %-10s | %-15s | %-15s\n', 'x1', 'x2', 'f(x1)', 'f(x2)');
  fprintf('-------------------------------------------------------------\n');

  x1 = a;
  while x1 < b
    x2 = min(x1 + dx, b);          % Evita pasarse
```

```
    fx1 = f(x1);
    fx2 = f(x2);



    % Determinamos el string del signo para f(x1)
    if fx1 > 0
        s1 = '(+)';
    elseif fx1 < 0
        s1 = '(-)';
    else
        s1 = '(0)';
    end



    % Determinamos el string del signo para f(x2)
    if fx2 > 0
        s2 = '(+)';
    elseif fx2 < 0
        s2 = '(-)';
    else
        s2 = '(0)';
    end



    str_f1 = sprintf('%8.2e %s', fx1, s1);
    str_f2 = sprintf('%8.2e %s', fx2, s2);



    if fx1 * fx2 < 0
      intervalos = [intervalos; x1 x2];
    end



    fprintf('%10.4f | %10.4f | %15s | %15s\n', x1, x2, str_f1, str_f2);



    x1 = x2;
  end
  fprintf('-------------------------------------------------------------\n');
end
```

interpolacionLineal.m

```
function [raiz, iter, historial] = interpolacionLineal(f, a, b, tol,
max_iter)

  if nargin < 4 || isempty(tol)
    tol = 1e-5;
  endif
  if nargin < 5 || isempty(max_iter)
    max_iter = 100;
  endif

  % Verificación de raíces exactas en los extremos
  if f(a) == 0
    fprintf('\nEl extremo a = %f es exactamente una raiz.\n', a);
    raiz = a; iter = 0; historial = [0, a, b, a, 0, 0]; return;
  endif
  if f(b) == 0
    fprintf('\nEl extremo b = %f es exactamente una raiz.\n', b);
    raiz = b; iter = 0; historial = [0, a, b, b, 0, 0]; return;
  endif

  % Verificación de Condiciones de Fourier
  fprintf('\n--- Verificacion de Condiciones de Fourier en [%.4f, %.4f] ---
\n', a, b);

  if f(a) * f(b) > 0
    error('No se cumple Bolzano: f(a) y f(b) tienen el mismo signo.');
  else
    fprintf('Bolzano: Cumple (existe al menos una raiz).\n');
  endif

  % Evaluacion numerica de derivadas para verificar Fourier
  n_pts = 100;
  x_eval = linspace(a, b, n_pts);
  h = max(abs(b - a) * 1e-5, 1e-6);

  df_vals = (arrayfun(f, x_eval + h) - arrayfun(f, x_eval - h)) ./ (2 * h);
  d2f_vals = (arrayfun(f, x_eval + h) - 2 * arrayfun(f, x_eval) +
arrayfun(f, x_eval - h)) ./ (h^2);

  if min(df_vals) * max(df_vals) > 0
    fprintf('Monotonia: Cumple (f''(x) no cambia de signo, raiz unica).\n');
  else
    fprintf('Monotonia: No cumple (f''(x) cambia de signo en el intervalo).
\n');
  endif
```

```
  if min(d2f_vals) * max(d2f_vals) > 0
    fprintf('Concavidad: Cumple (f''''(x) no cambia de signo, concavidad
constante).\n');
  else
    fprintf('Concavidad: No cumple (f''''(x) cambia de signo, posible punto
de inflexion).\n');
  endif

  d2f_a = (f(a + h) - 2 * f(a) + f(a - h)) / (h^2);
  d2f_b = (f(b + h) - 2 * f(b) + f(b - h)) / (h^2);

  if f(a) * d2f_a > 0
    fprintf('Punto fijo de Fourier: x = %.4f (f(a)*f''''(a) > 0).\n', a);
  elseif f(b) * d2f_b > 0
    fprintf('Punto fijo de Fourier: x = %.4f (f(b)*f''''(b) > 0).\n', b);
  else
    fprintf('Punto fijo de Fourier: No determinado estrictamente.\n');
  endif
  fprintf('-----------------------------------------------------------------
\n\n');

  iter = 0;
  x_act = a; % Punto de referencia inicial
  error_act = tol + 1; % Fuerza a entrar al bucle en la primera vuelta


  % Preasignamos memoria para el historial (iter, a, b, x_act, f(x_act),
error_act)
  historial = zeros(max_iter, 6);

  while error_act > tol && iter < max_iter
    x_ant = x_act;

    % Formula de la secante / interpolacion lineal
    calculoSup = a * f(b) - b * f(a);
    calculoInf = f(b) - f(a);
    x_act = calculoSup / calculoInf;

    iter = iter + 1;

    % En la primera iteracion no se compara contra el valor inicial 'a'
    if iter > 1
      error_act = abs(x_act - x_ant);
      historial(iter, :) = [iter, a, b, x_act, f(x_act), error_act];
    else
      historial(iter, :) = [iter, a, b, x_act, f(x_act), NaN];
    endif
```

```
    % Verificacion de raiz exacta
    if f(x_act) == 0
      break;
    % Actualizacion del intervalo conservando el cambio de signo
    elseif f(a) * f(x_act) < 0
      b = x_act;
    else
      a = x_act;
    endif
  endwhile

  if iter >= max_iter
    warning('Se alcanzó el máximo de iteraciones sin llegar a la
tolerancia.');
  endif


  % Imprimimos la tabla con el historial al finalizar
  historial = historial(1:iter, :);
  fprintf('\nMetodo de Interpolacion Lineal\n');
  fprintf('%-5s | %-12s | %-12s | %-12s | %-12s | %-12s\n', 'Iter', 'a',
'b', 'x_k', 'f(x_k)', 'Error Est.');

fprintf('------------------------------------------------------------------------
-------------\n');
  for i = 1:size(historial, 1)
    if isnan(historial(i, 6))
      fprintf('%-5d | %12.6f | %12.6f | %12.6f | %12.6f | %12s\n',
historial(i, 1), historial(i, 2), historial(i, 3), historial(i, 4),
historial(i, 5), '-');
    else
      fprintf('%-5d | %12.6f | %12.6f | %12.6f | %12.6f | %12.6e\n',
historial(i, 1), historial(i, 2), historial(i, 3), historial(i, 4),
historial(i, 5), historial(i, 6));
    endif
  endfor
  fprintf('\nRaiz aproximada: x = %f (en %d iteraciones)\n', x_act, iter);

  raiz = x_act;
endfunction
```

newtonraphson.m

```
function [x, it] = NewtonRaphson(f,a,b, tol)
  pkg load symbolic
  warning('off', 'OctSymPy:sym:rationalapprox');
  1;
```

```
  xs = sym('xs');
  dfs = diff(f(xs), xs);
  d2fs = diff(f(xs), xs, 2);
  df = matlabFunction(dfs, 'vars', xs);
  d2f = matlabFunction(d2fs, 'vars', xs);

  t = linspace(a, b, 2001);


  %--- Verificación de Condiciones de Fourier ---
  fprintf('\n--- Verificacion de Condiciones de Fourier en [%.4f, %.4f] ---
\n', a, b);

  % Bolzano
  if f(a) * f(b) > 0
    error('No se cumple Bolzano: f(a) y f(b) tienen el mismo signo.');
  else
    fprintf('Bolzano: Cumple (existe al menos una raiz).\n');
  endif

  if all(df(t) > 0 || df(t) < 0)
    fprintf('Monotonia: Cumple (f''(x) no cambia de signo, raiz unica).\n');
  else
    fprintf('Monotonia: No cumple (f''(x) cambia de signo en el intervalo).
\n');
  endif

  if all(d2f(t) > 0 || d2f(t) < 0)
    fprintf('Concavidad: Cumple (f''''(x) no cambia de signo, concavidad
constante).\n');
  else
    fprintf('Concavidad: No cumple (f''''(x) cambia de signo, posible punto
de inflexion).\n');
  endif

  % Condicion de Fourier para determinar el punto fijo
  if f(a) * d2f(a) > 0
    fprintf('Punto fijo de Fourier: x = %.4f (f(a)*f''''(a) > 0).\n', a);
    x = a;
  elseif f(b) * d2f(b) > 0
    fprintf('Punto fijo de Fourier: x = %.4f (f(b)*f''''(b) > 0).\n', b);
    x = b;
  else
    fprintf('Punto fijo de Fourier: No determinado estrictamente.\n');
    x = a;
  endif
  fprintf('----------------------------------------------------------------
\n\n');
```

```
  % Iteracion de Newton Raphson
  printf('It.  | %-8s | %-8s | %-9s | %-8s\n', 'xn', 'f(xn)', 'df(xn)',
'Error');
  printf('-------------------------------------------------------\n');

  for it = 1:100
    xn = x - f(x) / df(x);
    error = abs(xn - x);
    printf("x%d  | %.6f | %.6f | %.6f | %.6f\n", it, xn, f(xn), df(xn),
error);
    x = xn;
    if error < tol, break; end
  endfor

  printf("\nRaiz aproximada %.6f en %d iteraciones\n", x, it);
endfunction
```

Resultado de la ejecución

Para realizar las comparaciones de los métodos usaremos la función

$$f(h)=\frac{(pi\ *\ h^2*\ (9\ -\ h))}{3}-\ 30$$

del ejercicio 1 y la función $f(x)=\ 8*x\ +\ 0{,}3*x^2-\ 0{,}0013*x^3-\ 372$

del ejercicio 2 de la guía de laboratorio 3

Evaluación Ejercicio 1:

Método de tanteos(Igual para ambos métodos):

```
------------------------------------------------------------
PASO 2: METODO DE TANTEO
------------------------------------------------------------
Incremento dx para el tanteo [1]: 0.5

>>> BUSQUEDA DE RAICES POR TANTEO <<<
x1         | x2         | f(x1)            | f(x2)
-------------------------------------------------------------
   -3.0000 |    -2.5000 |    8.31e+01 (+) |    4.53e+01 (+)
   -2.5000 |    -2.0000 |    4.53e+01 (+) |    1.61e+01 (+)
   -2.0000 |    -1.5000 |    1.61e+01 (+) |   -5.26e+00 (-)
   -1.5000 |    -1.0000 |   -5.26e+00 (-) |   -1.95e+01 (-)
   -1.0000 |    -0.5000 |   -1.95e+01 (-) |   -2.75e+01 (-)
   -0.5000 |     0.0000 |   -2.75e+01 (-) |   -3.00e+01 (-)
    0.0000 |     0.5000 |   -3.00e+01 (-) |   -2.78e+01 (-)
    0.5000 |     1.0000 |   -2.78e+01 (-) |   -2.16e+01 (-)
    1.0000 |     1.5000 |   -2.16e+01 (-) |   -1.23e+01 (-)
    1.5000 |     2.0000 |   -1.23e+01 (-) |   -6.78e-01 (-)
    2.0000 |     2.5000 |   -6.78e-01 (-) |    1.25e+01 (+)
    2.5000 |     3.0000 |    1.25e+01 (+) |    2.65e+01 (+)
    3.0000 |     3.5000 |    2.65e+01 (+) |    4.06e+01 (+)
    3.5000 |     4.0000 |    4.06e+01 (+) |    5.38e+01 (+)
    4.0000 |     4.5000 |    5.38e+01 (+) |    6.54e+01 (+)
    4.5000 |     5.0000 |    6.54e+01 (+) |    7.47e+01 (+)
    5.0000 |     5.5000 |    7.47e+01 (+) |    8.09e+01 (+)
    5.5000 |     6.0000 |    8.09e+01 (+) |    8.31e+01 (+)
    6.0000 |     6.5000 |    8.31e+01 (+) |    8.06e+01 (+)
    6.5000 |     7.0000 |    8.06e+01 (+) |    7.26e+01 (+)
    7.0000 |     7.5000 |    7.26e+01 (+) |    5.84e+01 (+)
    7.5000 |     8.0000 |    5.84e+01 (+) |    3.70e+01 (+)
    8.0000 |     8.5000 |    3.70e+01 (+) |    7.83e+00 (+)
    8.5000 |     9.0000 |    7.83e+00 (+) |   -3.00e+01 (-)
    9.0000 |     9.5000 |   -3.00e+01 (-) |   -7.73e+01 (-)
    9.5000 |    10.0000 |   -7.73e+01 (-) |   -1.35e+02 (-)
-------------------------------------------------------------
```

Figura 1: Método de tanteos Ejercicio 1

Para ambos métodos usaremos una aproximación con un error absoluto de 1e-3 < 0

Nota: Para este ejercicio se tiene en cuenta 1 de los intervalos[2, 2.5] debido a que se representa un tanque esferico que no puede tener contenido negativo o superior a su capacidad total.

Resultado Interpolacion Lineal

```
>>> METODO 1: INTERPOLACION LINEAL <<<

--- Verificacion de Condiciones de Fourier en [2.0000, 2.5000] ---
Bolzano: Cumple (existe al menos una raiz).
Monotonia: Cumple (f'(x) no cambia de signo, raiz unica).
Concavidad: Cumple (f''(x) no cambia de signo, concavidad constante).
Punto fijo de Fourier: x = 2.5000 (f(b)*f''(b) > 0).
-----------------------------------------------------------------


Metodo de Interpolacion Lineal
Iter  | a           | b           | x_k         | f(x_k)      | Error Est.
-------------------------------------------------------------------------------
1     |    2.000000 |    2.500000 |    2.025659 |   -0.031537 |          -
2     |    2.025659 |    2.500000 |    2.026849 |   -0.001443 | 1.189701e-03
3     |    2.026849 |    2.500000 |    2.026903 |   -0.000066 | 5.441985e-05

Raiz aproximada: x = 2.026903 (en 3 iteraciones)
```

Figura 2: Resultado Interpolación Lineal

Resultado Newton-Raphson

```
>>> METODO 2: NEWTON-RAPHSON <<<
Symbolic pkg v3.2.2: Python communication link active, SymPy v1.10.1.

--- Verificacion de Condiciones de Fourier en [2.0000, 2.5000] ---
Bolzano: Cumple (existe al menos una raiz).
Monotonia: Cumple (f'(x) no cambia de signo, raiz unica).
Concavidad: Cumple (f''(x) no cambia de signo, concavidad constante).
Punto fijo de Fourier: x = 2.5000 (f(b)*f''(b) > 0).
-----------------------------------------------------------------

It.  | xn        | f(xn)     | df(xn)     | Error
-------------------------------------------------------------
x1   | 2.043729 | 0.426484 | 25.401492 | 0.456271
x2   | 2.026939 | 0.000852 | 25.299727 | 0.016790
x3   | 2.026906 | 0.000000 | 25.299521 | 0.000034

Raiz aproximada 2.026906 en 3 iteraciones
```

Figura 3: Resultado Newton Raphson

Comparación de resultados:

```
------------------------------------------------------------
RESULTADOS FINALES Y COMPARACION
------------------------------------------------------------
Metodo                   Raiz           Iter     tic/toc (s)     cputime (s)
Interpolacion lineal     2.02690312     3        7.54le-03       1.562e-02
Newton-Raphson           2.02690573     3        8.93le+00       8.594e-01

Detalle:
  Variable   : h
  Intervalo  : [2.0000, 2.5000]
  Tolerancia : 1.0e-03
  f(Raiz IL) : -6.60e-05
  f(Raiz NR) : 3.47e-09
------------------------------------------------------------
```

Figura 4: Resultados finales y comparación ejercicio 1

Evaluacion Ejercicio 2:

Metodo de tanteos(Igual para ambos métodos):

```
------------------------------------------------------------
PASO 2: METODO DE TANTEO
------------------------------------------------------------
Incremento dx para el tanteo [25]: 10

>>> BUSQUEDA DE RAICES POR TANTEO <<<
x1          | x2          | f(x1)              | f(x2)
------------------------------------------------------------
     0.0000 |     10.0000 |   -3.72e+02 (-) |   -2.63e+02 (-)
    10.0000 |     20.0000 |   -2.63e+02 (-) |   -1.02e+02 (-)
    20.0000 |     30.0000 |   -1.02e+02 (-) |    1.03e+02 (+)
    30.0000 |     40.0000 |    1.03e+02 (+) |    3.45e+02 (+)
    40.0000 |     50.0000 |    3.45e+02 (+) |    6.16e+02 (+)
    50.0000 |     60.0000 |    6.16e+02 (+) |    9.07e+02 (+)
    60.0000 |     70.0000 |    9.07e+02 (+) |    1.21e+03 (+)
    70.0000 |     80.0000 |    1.21e+03 (+) |    1.52e+03 (+)
    80.0000 |     90.0000 |    1.52e+03 (+) |    1.83e+03 (+)
    90.0000 |    100.0000 |    1.83e+03 (+) |    2.13e+03 (+)
   100.0000 |    110.0000 |    2.13e+03 (+) |    2.41e+03 (+)
   110.0000 |    120.0000 |    2.41e+03 (+) |    2.66e+03 (+)
   120.0000 |    130.0000 |    2.66e+03 (+) |    2.88e+03 (+)
   130.0000 |    140.0000 |    2.88e+03 (+) |    3.06e+03 (+)
   140.0000 |    150.0000 |    3.06e+03 (+) |    3.19e+03 (+)
   150.0000 |    160.0000 |    3.19e+03 (+) |    3.26e+03 (+)
   160.0000 |    170.0000 |    3.26e+03 (+) |    3.27e+03 (+)
   170.0000 |    180.0000 |    3.27e+03 (+) |    3.21e+03 (+)
   180.0000 |    190.0000 |    3.21e+03 (+) |    3.06e+03 (+)
   190.0000 |    200.0000 |    3.06e+03 (+) |    2.83e+03 (+)
   200.0000 |    210.0000 |    2.83e+03 (+) |    2.50e+03 (+)
   210.0000 |    220.0000 |    2.50e+03 (+) |    2.07e+03 (+)
   220.0000 |    230.0000 |    2.07e+03 (+) |    1.52e+03 (+)
   230.0000 |    240.0000 |    1.52e+03 (+) |    8.57e+02 (+)
   240.0000 |    250.0000 |    8.57e+02 (+) |    6.55e+01 (+)
   250.0000 |    260.0000 |    6.55e+01 (+) |   -8.61e+02 (-)
   260.0000 |    270.0000 |   -8.61e+02 (-) |   -1.93e+03 (-)
   270.0000 |    280.0000 |   -1.93e+03 (-) |   -3.15e+03 (-)
   280.0000 |    290.0000 |   -3.15e+03 (-) |   -4.53e+03 (-)
   290.0000 |    300.0000 |   -4.53e+03 (-) |   -6.07e+03 (-)
------------------------------------------------------------
```

Figura 5: Método de tanteos Ejercicio 2

Para ambos métodos usaremos una aproximación con un error absoluto de 1e-3 < 0

Resultados Interpolación Lineal:

Intervalo [250, 260]

```
>>> METODO 1: INTERPOLACION LINEAL <<<

--- Verificacion de Condiciones de Fourier en [250.0000, 260.0000] ---
Bolzano: Cumple (existe al menos una raiz).
Monotonia: Cumple (f'(x) no cambia de signo, raiz unica).
Concavidad: Cumple (f''(x) no cambia de signo, concavidad constante).
Punto fijo de Fourier: x = 260.0000 (f(b)*f''(b) > 0).
----------------------------------------------------------------


Metodo de Interpolacion Lineal
Iter  | a            | b            | x_k          | f(x_k)       | Error Est.
--------------------------------------------------------------------------------
1     |   250.000000 |   260.000000 |   250.707114 |     4.526980 |          -
2     |   250.707114 |   260.000000 |   250.755730 |     0.310055 | 4.861596e-02
3     |   250.755730 |   260.000000 |   250.759059 |     0.021223 | 3.328531e-03
4     |   250.759059 |   260.000000 |   250.759287 |     0.001453 | 2.278252e-04

Raiz aproximada: x = 250.759287 (en 4 iteraciones)
```

Figura 6: Interpolación Lineal - Intervalo [250, 260]

Intervalo [20, 30]

```
>>> METODO 1: INTERPOLACION LINEAL <<<

--- Verificacion de Condiciones de Fourier en [20.0000, 30.0000] ---
Bolzano: Cumple (existe al menos una raiz).
Monotonia: Cumple (f'(x) no cambia de signo, raiz unica).
Concavidad: Cumple (f''(x) no cambia de signo, concavidad constante).
Punto fijo de Fourier: x = 30.0000 (f(b)*f''(b) > 0).
----------------------------------------------------------------


Metodo de Interpolacion Lineal
Iter  | a            | b            | x_k          | f(x_k)       | Error Est.
--------------------------------------------------------------------------------
1     |    20.000000 |    30.000000 |    24.987823 |    -5.062866 |          -
2     |    24.987823 |    30.000000 |    25.222866 |    -0.219768 | 2.350436e-01
3     |    25.222866 |    30.000000 |    25.233047 |    -0.009484 | 1.018099e-02
4     |    25.233047 |    30.000000 |    25.233487 |    -0.000409 | 4.392959e-04

Raiz aproximada: x = 25.233487 (en 4 iteraciones)
```

Figura 7: Interpolación Lineal - Intervalo [20, 30]

Resultados Newton-Raphson

Intervalo [250, 260]

```
>>> METODO 2: NEWTON-RAPHSON <<<

--- Verificacion de Condiciones de Fourier en [250.0000, 260.0000] ---
Bolzano: Cumple (existe al menos una raiz).
Monotonia: Cumple (f'(x) no cambia de signo, raiz unica).
Concavidad: Cumple (f''(x) no cambia de signo, concavidad constante).
Punto fijo de Fourier: x = 260.0000 (f(b)*f''(b) > 0).
-----------------------------------------------------------------

It.  | xn        | f(xn)     | df(xn)     | Error
-------------------------------------------------------------
x1   | 251.360899 | -52.450518 | -87.594437 | 8.639101
x2   | 250.762111 | -0.243643 | -86.781115 | 0.598788
x3   | 250.759303 | -0.000005 | -86.777308 | 0.002808
x4   | 250.759303 | -0.000000 | -86.777308 | 0.000000

Raiz aproximada 250.759303 en 4 iteraciones
```

Figura 8: Newton Raphson - Intervalo [250, 260]

Intervalo [20, 30]

```
>>> METODO 2: NEWTON-RAPHSON <<<

--- Verificacion de Condiciones de Fourier en [20.0000, 30.0000] ---
Bolzano: Cumple (existe al menos una raiz).
Monotonia: Cumple (f'(x) no cambia de signo, raiz unica).
Concavidad: Cumple (f''(x) no cambia de signo, concavidad constante).
Punto fijo de Fourier: x = 30.0000 (f(b)*f''(b) > 0).
-----------------------------------------------------------------

It.  | xn        | f(xn)     | df(xn)     | Error
-------------------------------------------------------------
x1   | 25.424633 | 3.955433 | 20.733773 | 4.575367
x2   | 25.233861 | 0.007319 | 20.657000 | 0.190772
x3   | 25.233506 | 0.000000 | 20.656857 | 0.000354

Raiz aproximada 25.233506 en 3 iteraciones
```

Figura 9: Newton Raphson - Intervalo [20, 30]

Comparación de resultados:

Intervalo[250,260]

```
------------------------------------------------------------
RESULTADOS FINALES Y COMPARACION
------------------------------------------------------------
Metodo                   Raiz           Iter     tic/toc (s)    cputime (s)
Interpolacion lineal     250.75928665   4        6.081e-03      3.125e-02
Newton-Raphson           250.75930339   4        3.299e-01      3.125e-01

Detalle:
  Variable   : x
  Intervalo  : [250.0000, 260.0000]
  Tolerancia : 1.0e-03
  f(Raiz IL) : 1.45e-03
  f(Raiz NR) : -3.64e-12
------------------------------------------------------------
```

Figura 10: Ejercicio 2 - Resultados finales y comparación intervalo [250, 260]

Intervalo [20,30]

```
------------------------------------------------------------
RESULTADOS FINALES Y COMPARACION
------------------------------------------------------------
Metodo                   Raiz           Iter     tic/toc (s)    cputime (s)
Interpolacion lineal     25.23348659    4        6.289e-03      0.000e+00
Newton-Raphson           25.23350639    3        3.003e-01      2.969e-01

Detalle:
  Variable   : x
  Intervalo  : [20.0000, 30.0000]
  Tolerancia : 1.0e-03
  f(Raiz IL) : -4.09e-04
  f(Raiz NR) : 2.53e-08
------------------------------------------------------------
```

Figura 11: Ejercicio 2 - Resultados finales y comparación intervalo [20, 30]

Gráficos

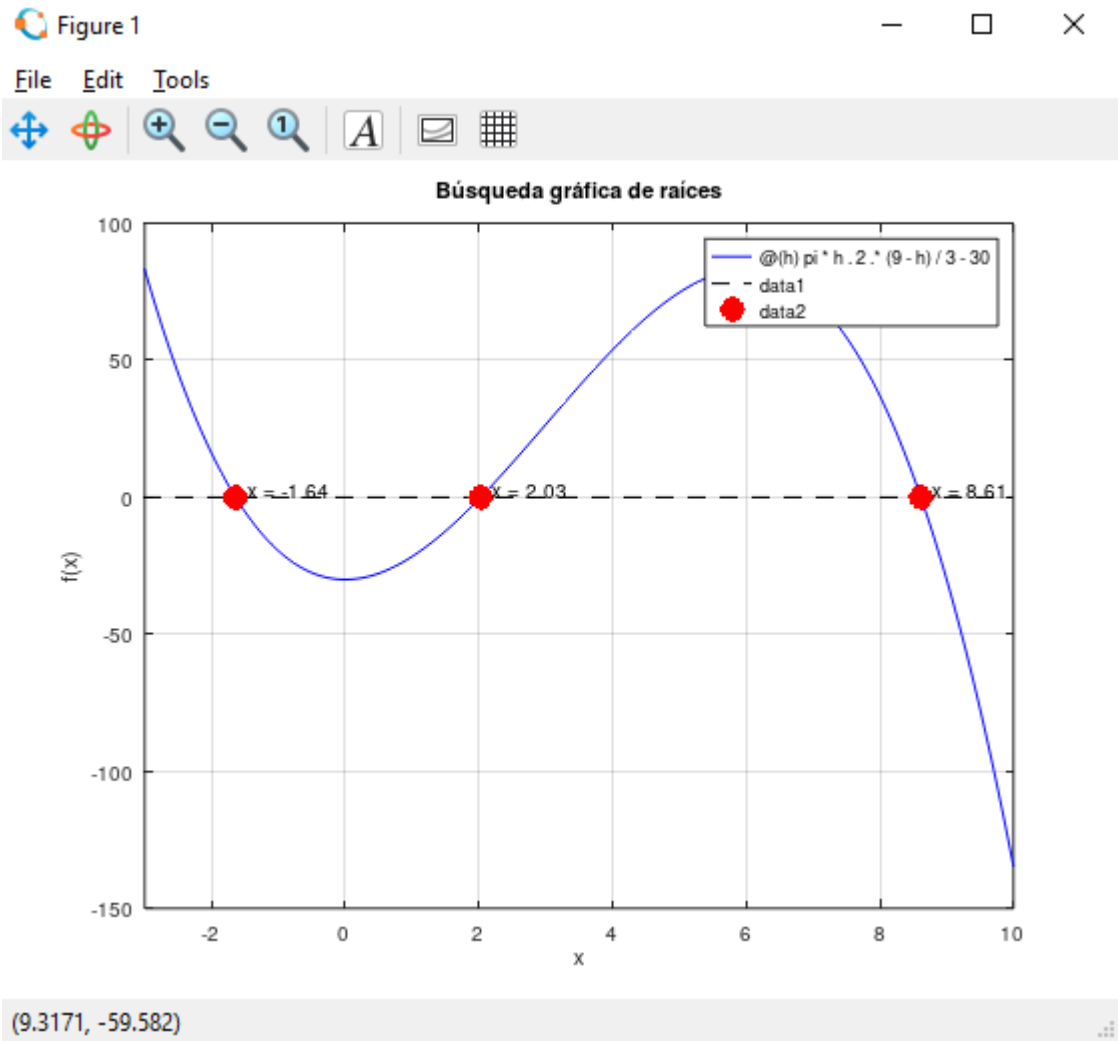


Figura 12: Gráfico - Ejercicio 1

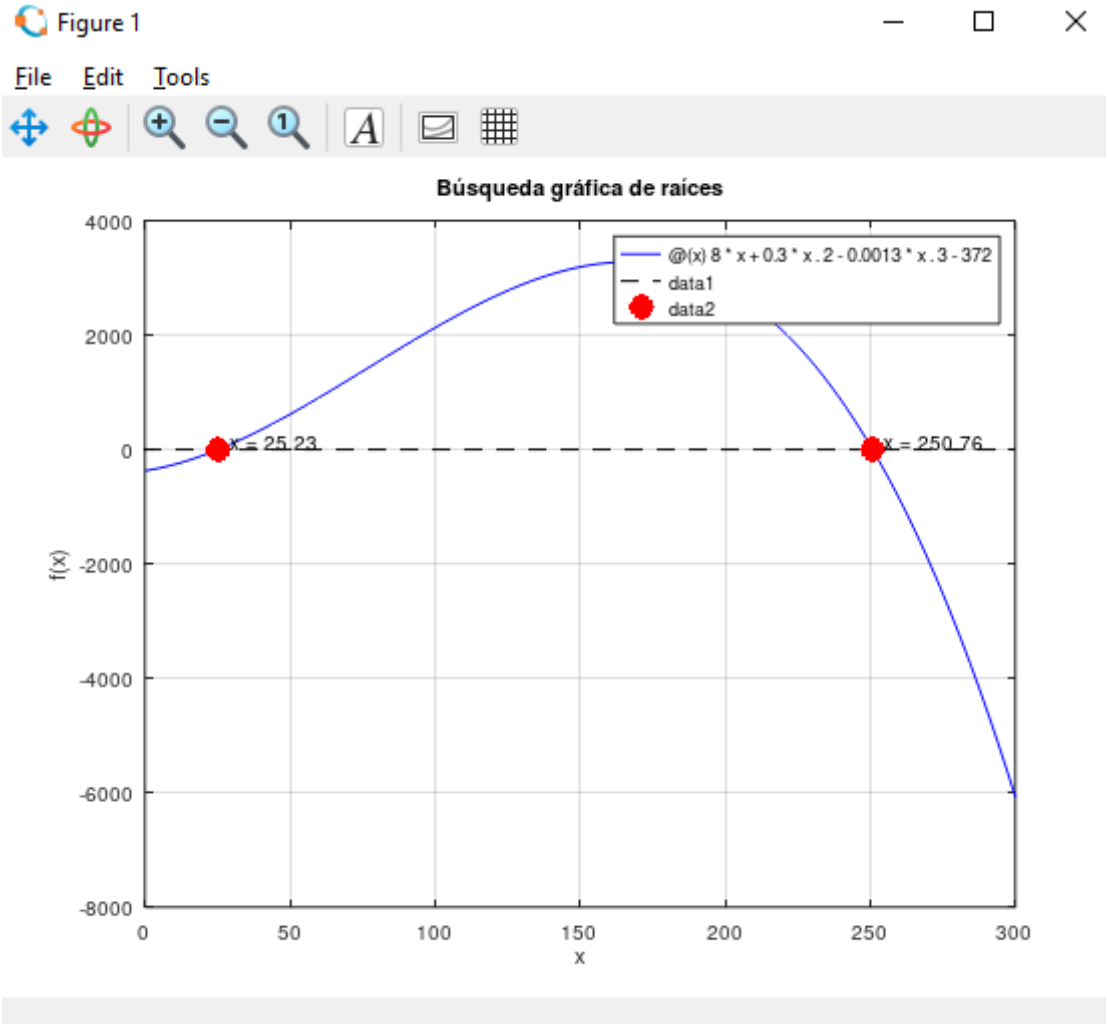


Figura 13: Grafico - Ejercicio 2

Conclusión

En el presente informe se abordó la resolución numérica de ecuaciones de la forma f(x) = 0 aplicando los métodos de Interpolación Lineal y Newton-Raphson sobre las funciones de la Guía de Laboratorio N° 3. Ambos métodos lograron aproximar exitosamente las raíces buscadas cumpliendo con la tolerancia fijada de E < 0.001.

Al analizar los resultados numéricos obtenidos en los programas de GNU Octave, se confirman en la práctica las diferencias teóricas discutidas en el desarrollo:

Ejercicio 1 (Tanque esférico): Partiendo del intervalo [2; 2.5], ambos métodos alcanzaron la raíz aproximada en 3 iteraciones (h aproximado a 2.0269 m). Sin embargo, Newton-Raphson demostró ser significativamente más preciso al lograr un residuo f(Raíz) = 3.47e-09) más cercano a cero que el de Interpolación Lineal (f(Raíz) = -6.60e-05).

Ejercicio 2 (Producción): Se repite la misma tendencia del ejercicio anterior, en el intervalo [20; 30], para este ejercicio Newton-Raphson necesitó 3 iteraciones dejando un residuo de 2.53e-08, mientras que Interpolación Lineal requirió 4 iteraciones con un residuo de -4.09e-04. La conclusión a la que llegamos es que esto se debe al tipo de convergencia de Newton-Raphson, al ser de tipo cuadrática este método duplica aproximadamente las cifras exactas en cada paso, mientras que Interpolación Lineal reduce el error de forma proporcional y lineal.

No obstante, si bien la convergencia de Newton-Raphson le permite afinar el margen de error mucho más rápido por paso, esto exige evaluar en cada iteración tanto la función f(x) como su primera derivada f'(x). En cambio, Interpolación Lineal, al ser un método de convergencia lineal, realiza un cálculo más simple evaluando únicamente f(x), lo que se refleja en un tiempo de ejecución por iteración sumamente bajo (del orden de 10^-3 segundos con tic/toc).

El criterio de selección del método varía en torno al tipo de problema que se esté tratando:

- Interpolación Lineal es más conveniente cuando se cuenta con un intervalo con cambio de signo (f(a) * f(b) < 0) y se prioriza la seguridad de convergencia, ya que al ser un método cerrado garantiza mantener la raíz atrapada sin la necesidad de calcular derivadas.
- Newton-Raphson es la alternativa más conveniente cuando se cuenta con un buen punto inicial X_0 (que verifique el criterio de Fourier) y la derivada es accesible, permitiendo alcanzar una precisión extrema con un número mínimo de iteraciones.

Bibliografía

[1] Chapra, S. C. y Canale, R. P. *Métodos numéricos para ingenieros*. McGraw-Hill. Capítulos 5 y 6: métodos cerrados y métodos abiertos.

[2] Burden, R. L. y Faires, J. D. *Análisis numérico*. Capítulo sobre soluciones de ecuaciones en una variable.

[3] Cátedra de Métodos Computacionales (2026). *Guía de Laboratorio N° 3*. Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.

[4] Eaton, J. W. y otros. *GNU Octave Documentation*. https://docs.octave.org